MECH6480: Computational Fluid Dynamics

> Alternative Approaches to CFD

applied-class-w-12

Overview

The primary aim for today is to measure the vortex-shedding Strouhal number behind a cylinder at $Re = 100$ with our Lattice Boltzmann code and compare it with experimental observations. Use the remaining time for your assignment. The **Optional** items at the end are unassessed.

Background: vortex shedding

This will be familiar to those who studied MECH3780 and used commercial software to study vortex shedding; today we do it with our own Lattice Boltzmann code (do not worry if you did not do MECH3780 at UQ). At low $Re = UD/\nu \lesssim 5$–15 the flow around a cylinder does not separate. From this range up to $Re \approx 40$ the flow separates and a fixed pair of *Föppl* vortices sits in the wake. For $40 \lesssim Re \lesssim 150$ a laminar vortex street forms behind the cylinder.

The shedding frequency is non-dimensionalised with the *Strouhal* number,

$$St = \frac{fD}{U}, \tag{1}$$

where f is the shedding frequency, U the free-stream velocity and D the cylinder diameter. In the laminar range, Williamson (1988) fitted experiments as

$$St = -\frac{3.3265}{Re} + 0.1816 + 1.6 \times 10^{-4}\, Re, \qquad 49 < Re < 178, \tag{2}$$

which gives $St \approx 0.164$ at $Re = 100$.

Part I: For submission

(a) **Run the starter.** Download and inspect `wk12-lbm-starter.py` from Blackboard. It simulates flow over a cylinder with a *velocity inlet* and an *extrapolated (zero-gradient)* outlet, as in the lecture. Slip (symmetry) boundaries are already implemented on the north and south faces: find them in the code (`FULLSLIP`) and note that, after streaming, the populations on these rows have their wall-normal component reflected while keeping their tangential component. Run the code and watch the steady wake turn into a vortex street. *You do not need to run until it is completely finished.*

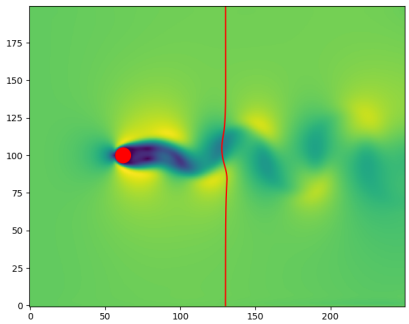


Figure 1: *Schematic of the domain generated with the starter code at* $Re = 100$*. The domain is* $L \times H = 25D \times 20D$*, with the cylinder of diameter* D *centred at* $(L/4, H/2)$*.*

(b) **Add a probe and a second subplot.** The code is set up for $Re = 100$. **Add** a history point at $(0.75L, 0.6H)$ (i.e. $2D$ above the wake centreline) and record the pressure $p = \rho c_s^2 = \rho/3$ every iteration. **Alter** the plotting so the figure has a second subplot showing the probe pressure against iteration number, with x-limits $(0, \texttt{N_ITER})$ and y-limits $(0.325, 0.340)$. At the **end** of the run, save the pressure trace to a csv file and save the two-subplot figure. Add your Git commit string and timestamp to the plot. *You may like to test on 'fewer' iterations to make sure it is working.*

Why off the centreline? Vortices pass alternately above and below the centreline, so a pressure probe exactly at $0.5H$ sees two pressure dips per shedding cycle and its FFT peaks at $2f$. Off the centreline the fundamental f dominates. (A probe of u_y is another clean option.)

Python running note: Live plotting (especially `streamplot`) can dominate the runtime. For the production run increase `plot_every` (e.g. `N_ITER//10`). **Start your 20,000-iteration run early**, then do part (c) on the reference trace while it runs.

(c) **Extract the shedding frequency.** Use the function below, which removes the mean, applies a Hann window and zero-pads the FFT before picking the peak. The `start` argument discards the start-up transient: choose it from the trace where the oscillations have become steady, and justify your choice.

Listing 1: *FFT of the probe signal to extract the dominant frequency.* `start` *is the iteration after which shedding is periodic.*

```python
import numpy as np
import scipy.fft as fft

def determine_frequency(pressure, start, n_pad=2**18):
    pressure = pressure[start:]
    mean_pressure = pressure - np.mean(pressure)
    window = np.hanning(len(mean_pressure))
    windowed_signal = mean_pressure * window
    spectrum = fft.rfft(windowed_signal, n=n_pad)
    freqs = fft.rfftfreq(n_pad)
    spectrum[freqs < 1e-4] = 0 # Remove numerical noise
    freq_max = freqs[np.argmax(np.abs(spectrum))]
    return freq_max

def strouhal(f, U = 0.05, D = 10):
    return f*D/U

def strouhal_empirical(Re):
    # Williamson (1988), valid for 49 < Re < 178
    return -3.3265/Re + 0.1816 + 1.6e-4*Re
```

Use this code on three traces in this order:

(i) **Verification:** apply the function (with `start=0`) to `np.sin(2*np.pi*8e-4*np.arange(20000))` and confirm it returns $\approx 8 \times 10^{-4}$.

(ii) **Reference trace:** apply it to the provided `pressure_history_centreline.csv` on Blackboard (columns: time step, pressure). First plot the pressure against time and determine an appropriate `start` to discard the transient. This data was taken at the centreline of the cylinder at $Re = 100$. Compare with the Williamson correlation, but keep in ind the earlier note about the centreline!

(iii) **Trace from your run:** apply it to your own trace and compare with the Williamson correlation.

For submission:

Submit to Gradescope:

1. Your figure of the simulation and pressure trace after 20,000 iterations, with time stamp and commit string (as a PNG file).
2. A St–Re plot of the Williamson curve ($49 < Re < 178$) with your $Re = 100$ point for comparison (as a PNG file).
3. Your modified Lattice Boltzmann code and the script used to compute the St–Re plot, including the frequency analysis of your pressure trace (as Python files).

Optional (unassessed tasks for extra practice)

(i) Run $Re \in \{60, 80, 100, 120, 140\}$ (or a selection) and add the points to your St–Re plot.

(ii) At $Re = 100$, place the cylinder in a channel of height $H_{channel} = 0.5H$ (still centred in y) using the no-slip wall boundary conditions from the lecture code. How does the shedding frequency change?

(iii) Try novel geometries (e.g. an aerofoil at an angle of attack) or several interacting cylinders.

If you are interested to test additional cases or looking for extra practice: Search for moving wall boundary conditions in the Timm Kruger textbook: *Lattice Boltzmann Methods – Principles and Practice* available at UQ library website. Implement a moving wall on the top boundary. Then try to simulate a lid-driven cavity flow and compare with the benchmark data provided for your first assignment. The moving wall condition should look like:

Listing 2: *Moving wall boundary condition.*

```
#5 Moving lid: add the wall momentum to the populations bounced off the top row
#NOTE: this would apply after performing standard bounceback!
f_prec[7, 1:-1, -1] -= RHO0*U_LID/6   # bounced from f5
f_prec[8, 1:-1, -1] += RHO0*U_LID/6   # bounced from f6
```

Another optional practice, is to implement the backwards facing step into the LBM code and compare with our FVM approach from earlier in semester.